

Software Standards and Guidelines

Author: Chiara Gillam

Version: 1.0.1

Last updated: 27/03/2026



Purpose & Scope

This document establishes the standards and guidelines for all software developed, maintained, or integrated by the team. Its purpose is to ensure consistency, quality, safety, and maintainability across all software components, regardless of project or contributor. It defines the minimum expectations for design, implementation, documentation, testing, and review activities.

These standards apply to all team members involved in software development, including developers, testers, integrators, and reviewers. The scope includes embedded firmware, application software, internal tools, and supporting scripts. Hardware design standards, system-level safety requirements, and third-party vendor documentation are outside the scope of this document and are covered in separate standards.

This document should be read in conjunction with all other Standards documents produced by MREx, and project-specific design documents. All “shall” statements are mandatory unless a formal deviation is approved.

Outline

Purpose & Scope.....	2
Outline.....	3
Normative References.....	5
Definitions & Acronyms.....	5
CAN specific.....	5
Others.....	5
Roles & Responsibilities.....	6
General Requirements.....	6
Detailed Standards / Guidelines.....	7
1. Coding standards.....	7
1.1 C++ / Arduino IDE style guide.....	7
2. CAN MREx Standards.....	10
2.1 Object Dictionary variables.....	10
2.2 SDOs.....	11
2.3 PDOs.....	11
2.4 EMCY.....	12
3. Design Standards.....	12
3.1 Design Principles.....	12
3.2 Design Documentation Requirements.....	12
3.3 Module Boundaries.....	13
3.4 Hardware Abstraction.....	13
3.5 State Machine Requirements.....	13
3.7 Timing and Non-Blocking Behaviour.....	13
3.8 Complexity Limits.....	13
3.9 Reuse and Shared Code.....	14
4. Testing Standards.....	14
4.1 General Testing Requirements.....	14
4.2 Test Case Standards.....	15
4.3 Integration and System Testing.....	15
4.4 Continuous Integration Requirements.....	16
4.5 Documentation Requirements for Testing.....	16
4.6 Test Artefact Storage.....	16
5. Review Standards & Merging.....	17
5.1 Merging.....	17
5.2 Review Assignment.....	17
5.3 Required Number of Reviewers.....	18
5.4 What Reviewers MUST Check.....	18
5.5 Expectations for Constructive Feedback.....	18
5.6 Requirements for Merging.....	19
5.7 Reviewer Responsibilities.....	19
6. Documentation Standards.....	19
6.1 Required Documentation for New Features.....	19

6.2 Updating Existing Documentation.....	20
6.3 Versioning Rules.....	20
6.4 Changelog Expectations.....	20
6.5 Documentation Storage Requirements.....	20
6.6 Documentation Quality Requirements.....	21
6.7 Documentation as a Living Artefact.....	21
7. Versioning & Release Standards.....	21
7.1 Semantic Versioning Rules.....	21
7.3 Branching Standards (General).....	22
7.4 Node Software Release & Branching Standards.....	22
Standards and Guidelines Version Control.....	23
Appendices (Templates, Examples, Checklists).....	24
References.....	25

Normative References

Other design documents that should be followed include:

- [Railway challenge Rules](#)
- [Railway challenge Technical Specifications](#)
-  PCB Standards and Guidelines
- <https://keepachangelog.com/en/1.1.0/>

Definitions & Acronyms

CAN specific

CAN - Controller Area Network, the physical and data link layer used for communication between nodes.

CAN MREx - The CAN bus application layer that is used at MREx for all communication between nodes.

CANopen - The communication protocol and device profile framework that CAN MREx is based on.

Node - A single Microcontroller / transceiver setup that can interact with other nodes on the bus.

OD / Object Dictionary – A structured table of all data objects on a Node that is accessible via SDO/PDO.

SDO (Service Data Object) – Point-to-point communication used for configuration and non-time-critical data transfer.

PDO (Process Data Object) – Real-time data transfer mechanism for transmitting operational values.

TPDO / RPDO – Transmit/Receive PDOs used for sending and receiving real-time data.

NMT (Network Management) – Protocol for controlling node states (e.g., Operational, Pre-Op, Stopped).

Heartbeat – Periodic message used to monitor node liveness and state.

EMCY (Emergency Message) – High-priority error message indicating a fault condition.

Others

MUNT / MREx Ultra Node Tool - The tool used to program the train.

MREx Dashboard - The tool used to view data that has been logged over a run in a user friendly UI

CI Checks - Continuous integration checks. They are tests / test cases that are tested through all changes of a project

Roles & Responsibilities

Within this team, software development is a collaborative effort shared across a small group of developers, with Section Leads providing final technical authority. All developers contribute to the design, implementation, testing, and review of software, ensuring that no work is completed in isolation and that every change benefits from multiple perspectives. Developers are responsible for producing clear, maintainable code, updating design documentation as features evolve, and performing initial testing to verify correctness before changes are submitted for review.

Peer review is a collective responsibility: every developer participates in reviewing each other's work to maintain quality, consistency, and adherence to the Software Standards & Guidelines. This shared review process ensures that knowledge is distributed across the team, reduces the risk of single-point expertise, and strengthens traceability between requirements, design, and implementation. Feedback is expected to be constructive, technically grounded, and focused on improving the overall quality of the system.

The Section Leads oversee the broader technical direction and have final approval authority for major design decisions, architectural changes, and high-impact modifications. While developers handle day-to-day implementation, the Leads ensure that the software remains coherent, maintainable, and aligned with project goals. The Lead also resolves ambiguities, clarifies requirements, and ensures that the team consistently follows the documented processes.

Together, this structure provides clear accountability while remaining lightweight and flexible. Developers own the work they produce, the team collectively maintains quality through peer review, and the Leads ensures technical consistency and final approval where required

General Requirements

All software must comply with the standards defined in this document. All “shall” and “must” statements are mandatory unless a formal deviation is approved. All software must comply with relevant external documents within the Normative References section. Software must prioritise safe behaviour under all operating conditions. Fault conditions must be handled explicitly and fail-safe behaviour must be defined. Safety-critical logic must include tests for invalid inputs, communication loss, and recovery. Code should be readable and maintainable for any experienced engineer who interacts with it. Software that requires a UI should be intuitive and easy to navigate and understand.

Detailed Standards / Guidelines

1. Coding standards

1.1 C++ / Arduino IDE style guide

1.1.1 File Header

Files MUST start off with the following comment to ensure that IP is protected and versions are being tracked.

```
/**
 * @file example.txt
 * @brief Short description of file.
 *
 * @details
 * Longer description of what this file does, what it uses,
 * what other files are connected, etc.
 *
 * @author Example Name
 *
 * @date      dd/mm/yyyy
 *
 * @version 1.0.0
 *
 * @organisation MREX
 *
 * @see other_file.c
 */
```

If you would like to include multiple authors use

- * @author First Author
- * @author Second Author

1.1.2 File Structure

All files should be in a folder named the same as your top module. (.ino file in arduino case).

1.1.3 File naming conventions

File names MUST be lowercase with underscores. Example:cm_emcy.c, motor_controller.ino

1.1.4 Include guard requirements

Header files MUST include guards. Guard names MUST be uppercase. Guard names MUST match the file name

```
#ifndef CM_EMCY_H
#define CM_EMCY_H

// header contents

#endif // CM_EMCY_H
```

1.1.5 Header Files

Header files MUST follow the same naming as the file it's attached to. All Pin definitions and function prototypes MUST be moved into here.

1.1.6 Function Documentation Requirements

All public functions MUST use Doxygen-style comments.

```
/**
 * @brief Push a minor error into the ring buffer.
 *
 * @param node Node ID that generated the error.
 * @param code 32-bit EMCY code.
 *
 * @return True if stored, false if buffer full.
 */
bool pushMinor(uint8_t node, uint32_t code);
```

Internal (static) functions MAY use shorter comments.

1.1.7 Indentation and Formatting

Code MUST be indented appropriately. Braces MUST follow the K & R style (braces on the same line as if, else, etc and closing bracket on own line).

```
if (bufferFull) {
    handleOverflow();
}
```

1.1.8 Commenting Expectations

Comments MUST explain why, not what. Complex logic MUST have inline comments. Hardware-specific behaviour MUST be documented.

1.1.9 Variable Naming

Local variables MUST use lowerCamelCase.

```
uint8_t myVariable;
static uint32_t thisVariable;
bool bufferFull;
```

Constants MUST use UPPER_CASE_WITH_UNDERSCORES.

```
#define MINOR_BUFFER_SIZE 16
static const uint8_t MAX_NODES = 32;
```

Global Variables MUST use snake_case and SHOULD be avoided unless necessary.


```
uint8_t system_state;
```

External Variables MUST use e_snake_case and SHOULD be avoided where necessary.

```
uint8_t e_system_state;
```

1.1.10 State Variables

State variables MUST have description followed by prev. It MUST follow naming conventions for global/local variables. Example:

```
static uint8_t statePrev = 0;
uint8_t state_prev = 0;
```

If a state variable is used only inside one function, it MUST be a static local inside that function.

Note: A static local variable is initialized only once, the first time the function runs. After that, it keeps its value between calls.

```
void HandleDirection() {
    static int dirPrev = 2; // This is the state variable

    if ((directionMode != dirPrev) && directionMode != 101) {
        dirprev = directionMode;
        executeSDOWrite(node_id, 1, 0x6060, 0x00,
sizeof(directionMode), &directionMode);
    }
}
```

1.1.11 Function Naming

Function names MUST clearly describe their purpose. They MUST use UpperCamelCase. Names MUST be verbs or verb phrases

```
bool PushMinor(uint8_t node, uint32_t code);
void InitEmcyBuffer(void);
uint8_t GetNextError(void);
```

If a file is not the top-level module and it contains a helper function that should not be accessed externally, that function must be declared static and prefixed with an underscore. This marks it as an internal, file-private function that must remain in that file.

Note: In Arduino projects, the top-level module is the primary .ino file that contains setup() and loop().

```
static void _ResetMinorBuffer(void);
```

1.1.12 Error Handling

All errors detected should be printed to serial monitor/console for ease of debugging.

1.1.13 TODOs

When there are features that need to be implemented in the future and you want to make a note of it, include your name and use the following syntax in C:

```
// TODO(Chiara): Implement debounce logic for button inputs

/*
 * TODO(Chiara):
 * - Add bounds checking
 * - Validate input range
 * - Improve documentation
 */
```

1.1.13 Forbidden Patterns

The following patterns **MUST NOT** be used unless explicitly approved:

Dynamic Memory Allocation

- malloc, free, new, delete
Embedded systems **MUST** avoid heap fragmentation.

Blocking Delays in Logic Functions

- delay() or long busy-wait loops
These **MUST** only appear in initialization or hardware-specific modules.

Magic Numbers

All numeric constants **MUST** be defined as macros or const variables.

Hidden Side Effects

Functions **MUST NOT** modify global state unless clearly documented.

2. CAN MREx Standards

2.1 Object Dictionary variables

2.1.1 Naming

OD variables **MUST** use od_ prefix followed by snake case. OD Names **MUST** describe the physical or logical signal represented in the Object Dictionary

```
uint8_t od_regen_brake = 0;
```

2.1.2 Scope and Usage

OD Variables **MUST** be used when:

- storing values read from the CANopen Object Dictionary
- preparing values to be written to the Object Dictionary
- mapping PDOs to internal variables
- representing any SDO-related data

OD Variables **MUST NOT** be used for:

- For internal logic not directly tied to the Object Dictionary
- For temporary computation variables

- For hardware pin reads/writes
- For UI or application-level state

2.1.3 In file Documentation Requirements

Every OD variable MUST include a comment describing:

- The OD index and subindex
- Brief Description
- Units (if applicable)
- Direction (read-only, write-only, read/write)
- PDO mapping (if applicable)

```
// OD 0x2030:01 - Regen brake request (0-1023). RW. Mapped to TPDO2.
uint16_t od_regen_brake;
```

2.1.4 Documentation Requirements

Every OD entry MUST be registered in the  CAN MREX Nodes Registry Registry. It is YOUR responsibility to ensure that the object dictionary variable you have created in the object dictionary, the node you are currently working on and whatever other nodes it gets routed to. If these variables change at all they MUST be documented in the registry.

2.1.5 Mutexes

If you have an object dictionary variable that can be changed by your own node and also by another node you must wrap it in a mutex.

2.2 SDOs

2.2.1 Read requests

When executing a read request, the variable you save into MUST be a 32 bit integer. If you want it of a different size use an intermediate variable to save it into with the postfix _32 and then typecast it into the size you would like.

```
heartBeatNode2_32 = ExecuteSDORead(node_id, 2, 0x1017, 0x00);
uint8_t heartBeatNode2 = (uint8_t)heartBeatNode2_32
```

2.2.2 Documentation Requirements

All SDOs MUST be mapped out in the  CAN MREX Nodes Registry . It is YOUR responsibility to ensure the SDO pairs are connected within this registry. Use Transmitting Read Request/Write request for the node that is transmitting the request and Receiving Read Request/Write request for the node that is receiving it under Communication type.

2.3 PDOs

2.3.1 Inhibit timer

Each PDO SHOULD have an inhibit timer if you are using the MarkTPDODirty() function to ensure the bus is not flooded with values.

2.3.2 Documentation Requirements

All TPDOs and RPDOs MUST be mapped out in the

 CAN MREx Nodes Registry . It is YOUR responsibility to ensure the TPDO and RPDO pairs/groups are connected within this registry.

2.4 EMCY

2.4.1 Emergency error codes

Emergency error codes SHOULD follow the following format:

Byte Value	Description
0x00	CAN MREx fault
0x01	Motor fault
0x02	Brake fault
0x03	Battery fault

2.5 The Main Loop

When Implementing a CAN MREx system you need to use the following structure for your to keep a code clean and maintainable

```
void loop() {
    OperatingMode mode =
static_cast<OperatingMode>(nodeOperatingMode); // Uses
nodeOperatingMode to decode what operating mode we're in

    switch (mode) {
        case MODE_STOPPED:
            StoppedMode();
            break;

        case MODE_PREOP:
            PreOpMode();
            break;

        case MODE_OPERATIONAL:
            OperationalMode();
    }
}
```

```

        break;

    default:
        StoppedMode(); // fail-safe
        break;
    }
}

```

This also means you need to declare this enum in your header.

```

// Enum for operating modes
enum OperatingMode : uint8_t {
    MODE_STOPPED        = 0x02,
    MODE_PREOP          = 0x80,
    MODE_OPERATIONAL    = 0x01
};

```

3. Design Standards

3.1 Design Principles

3.1.1 Single Responsibility

Each module, class, or function **MUST** have one clear purpose. A change in one feature should require changes in only one place.

3.1.2 Separation of Concerns

Hardware access, business logic, and communication logic **MUST** be separated into distinct modules or functions.

3.1.3 Modularity

Code **MUST** be organized into small, reusable components. Large monolithic functions **MUST** be broken into smaller units.

3.1.4 Explicit Data Flow

Data passed between modules **MUST** be clear and documented. Hidden dependencies **MUST** be avoided.

3.1.5 Fail-Safe Behaviour

All modules **MUST** define expected behaviour under fault conditions (e.g., communication loss, invalid inputs).

3.2 Design Documentation Requirements

Before implementation begins, developers **MUST** document:

- The purpose of the module
- Inputs and outputs
- Dependencies (hardware, CAN nodes, libraries)
- State variables and how they are updated
- Error handling behaviour
- Timing requirements (if applicable)

This documentation **MAY** be lightweight but **MUST** exist. This could simply be a flow diagram.

3.3 Module Boundaries

Each module **MUST** clearly define:

- What it owns
- What it exposes
- What it depends on
- What it must never modify

Modules **MUST NOT** reach into each other's internal state unless explicitly designed to do so.

3.4 Hardware Abstraction

Hardware-specific code (pin reads, ADC, PWM, GPIO) **MUST** be isolated from logic code.

This allows:

- Easier testing
- Easier simulation
- Easier hardware changes

Example:

`readBrakeSensor()` instead of calling `analogRead()` everywhere.

3.5 State Machine Requirements

If a module implements a state machine:

- States **MUST** be enumerated
- Transitions **MUST** be documented
- Invalid transitions **MUST** be handled safely
- State variables **MUST** follow naming conventions (e.g., `statePrev`)

(The operating modes are a state machine)

3.7 Timing and Non-Blocking Behaviour

All time-dependent logic **MUST**:

- Use non-blocking patterns (millis(), timers, state machines)
- Avoid delay() except in initialization
- Document timing assumptions (e.g., “runs every 100 ms”)

3.8 Complexity Limits

To maintain readability:

- Functions SHOULD NOT exceed ~50 lines
- Files SHOULD NOT exceed ~500 lines unless justified
- Deep nesting (more than 3 levels) SHOULD be avoided
- Cyclomatic complexity SHOULD remain low

High cyclomatic complexity (hard to read) example

```
void HandleBrake() {
    if (sensorOK) {
        if (pressure > threshold) {
            if (!override) {
                applyBrake();
            } else {
                if (manualMode) {
                    logOverride();
                }
            }
        }
    }
}
```

Low cyclomatic complexity (cleaner) example

```
void HandleBrake() {
    if (!sensorOK) return;
    if (pressure <= threshold) return;
    if (override && manualMode) {
        logOverride();
        return;
    }
    applyBrake();
}
```

3.9 Reuse and Shared Code

Common logic MUST be placed in shared modules rather than duplicated.

Examples:

- CRC functions
- CAN parsing
- Error handling
- Utility math functions

4. Testing Standards

4.1 General Testing Requirements

4.1.1 Test Coverage

All modules should include tests that verify:

- Normal behaviour
- Boundary conditions
- Error handling and fail-safe behaviour
- Timing-dependent behaviour (if applicable)

Safety-critical modules **MUST** include explicit tests for invalid inputs, communication loss, and recovery behaviour.

4.1.2 Pre-PR Testing Obligations

Before opening a Pull Request:

- All unit tests **MUST** pass on a clean environment
- Integration tests **MUST** pass for any module that interacts with hardware, CAN, or shared state before being moved to main.
- No PR may be opened with failing tests unless explicitly approved by the Integration Lead
- “Works on my machine” is **NOT** acceptable; tests **MUST** pass on a fresh environment

4.2 Test Case Standards

4.2.1 Test Case Format

Each test case **MUST** include:

- Objective – what is being tested
- Setup – hardware, inputs, state
- Procedure – steps to execute
- Expected Behaviour – measurable, objective pass/fail criteria

Test case names **MUST** follow:

TREQ_<module>_<number> (e.g., TREQ_EMCI_004)

4.2.2 Acceptance Criteria

Acceptance criteria **MUST**:

- Be defined in the design documentation before implementation
- Be objective and measurable
- Map directly to requirements (REQ → TREQ)
- Include timing constraints where relevant

4.2.3 Software-Specific Test Requirements

Certain modules require additional testing:

- CAN modules **MUST** test SDO, PDO, EMCY, NMT, and heartbeat-behaviour
- State machines **MUST** test all valid transitions and invalid transitions

- Hardware abstraction layers MUST test simulated and real hardware behaviour
- Safety-critical logic should include fault-injection tests

4.3 Integration and System Testing

4.3.1 Integration Test Expectations

Integration tests MUST verify:

- Communication between modules
- Shared state behaviour
- Timing interactions
- CAN message flow (send, receive, error handling)

Modules MUST NOT be considered complete until integration tests pass.

4.3.2 System-Level Validation

System tests should validate:

- End-to-end behaviour in realistic operating conditions
- Hardware-in-the-loop behaviour where applicable
- Stress conditions (high message load, rapid state changes)
- Fault conditions (node offline, invalid data, timeouts)

4.4 Continuous Integration Requirements

4.4.1 Required CI Checks

The CI pipeline MUST include:

- Unit tests
- Integration tests
- Static analysis
- Style and formatting checks
- Build verification

A PR MUST NOT be merged if any CI check fails.

4.4.2 CI Failure Handling

If CI fails:

- The PR author MUST investigate and fix the issue
- Failures MUST be documented in the PR discussion
- Overrides MAY be granted only by the Integration Lead and MUST be justified

4.5 Documentation Requirements for Testing

4.5.1 Testing in Design Documentation

All testing criteria MUST be defined in the design documentation before implementation begins.

Design documentation MUST include:

- Test cases for each requirement

- Acceptance criteria
- Expected behaviour under normal and fault conditions
- A Verification Matrix mapping REQ → TREQ

4.5.2 Updating Documentation After Testing

After testing:

- The Verification Matrix **MUST** be updated with pass/fail results
- Any failed test **MUST** include a documented root cause and resolution
- Documentation **MUST** be version-controlled and updated with each change

4.6 Test Artefact Storage

- All test logs, regression tests, and test harnesses **MUST** be stored in the repository
- Test data **MUST NOT** be stored locally or lost between versions
- Hardware test results **MUST** be recorded in a shared location accessible to the team

5. Review Standards & Merging

5.1 Merging

In order to merge your code it must be first reviewed. The review process consists of three steps.

Step one:

Read over your own code and compile it for the sake of ensuring that you haven't missed anything.

Step two:

Feed your code into a chatbot like chatgpt to see if it sees any mistakes. An example prompt could be:

Please review the following code for correctness, clarity and maintainability.

Check for:

- *naming conventions*
- *state variable rules*
- *function structure and complexity*
- *forbidden patterns (dynamic allocation, blocking delays, magic numbers)*
- *documentation and Doxygen requirements*

Explain any issues and suggest improvements. Here is the code:

Step three:

Make a pull request and nominate a reviewer or reviewers to look over the pull request. After they have approved it you can continue with merging.

5.2 Review Assignment

- The author of a Pull Request (PR) **MUST** nominate at least one reviewer at the time the PR is opened.

- The reviewer **MUST** be someone who did not directly write the code being reviewed.
- If the change affects multiple modules, the author **MUST** nominate reviewers from each relevant area.
- The Section Leads **MAY** assign additional reviewers if the change is high-impact or cross-cutting.

This ensures that every PR has a clear owner on both sides: the author and the reviewer.

5.3 Required Number of Reviewers

There **MUST** be a minimum of 1 reviewer for standard changes. There **SHOULD** be 2 reviewers for complex, safety-critical, or cross-module changes. There **MUST** be Section Lead approval for:

- Architectural changes
- Public API changes
- Changes affecting safety, reliability, or communication protocols (e.g., CAN, EMCY, SDO)
- Any change that modifies shared libraries or core infrastructure

5.4 What Reviewers **MUST** Check

5.3.1 **Correctness**

- Logic is correct and complete
- Edge cases are handled
- No undefined behaviour
- No race conditions or concurrency issues
- No hidden side effects

5.3.2 **Clarity**

- Code is readable and maintainable
- Naming follows the standards
- Functions are appropriately sized and scoped
- Comments explain why, not what

5.3.3 **Compliance**

- Follows coding standards
- Follows documentation standards (Doxygen, file headers, etc.)
- No forbidden patterns (dynamic allocation, blocking calls, magic numbers)
- Tests updated or added where required

5.3.4 **Impact**

- No unintended changes to unrelated modules
- No breaking changes to public APIs
- No regressions in behaviour

5.5 Expectations for Constructive Feedback

All feedback **MUST** be:

- Specific
- Actionable
- Respectful
- Focused on the code, not the author
- Supported by reasoning (e.g., standards, safety, maintainability)

Examples of constructive phrasing:

- “Consider renaming this variable for clarity.”
- “This logic may fail when the buffer wraps; can we add a check?”
- “This violates the no-dynamic-allocation rule; please refactor.”

Examples of unacceptable phrasing:

- “This is wrong.”
- “Why would you do it like this?”
- “Rewrite this.”

5.6 Requirements for Merging

A PR may only be merged when:

- All blocking comments are resolved
- All reviewers have approved
- The Section Lead/s has approved (if required e.g merging into main)
- CI checks (if any) have passed
- Documentation is updated
- Tests are updated or added
- Review process has been run through fully

The PR author is responsible for ensuring all conditions are met.

5.7 Reviewer Responsibilities

Reviewers MUST:

- Respond within a reasonable timeframe (e.g., 24–48 hours) or state whether they can’t respond within that timeframe.
- Provide clear, actionable feedback
- Re-review after changes are made
- Approve only when the code meets all standards
- Escalate to the Section Lead if:
 - There is disagreement
 - The change is high-risk
 - The design is unclear or incomplete

6. Documentation Standards

6.1 Required Documentation for New Features

Every new feature, subsystem, or design change **MUST** include an updated Design Report following the approved template. [Design Report Template](#) At minimum, the following sections **MUST** be completed or updated:

- Purpose & Scope — why the feature exists and its boundaries.
- System Overview — updated diagrams, interfaces, and dependencies.
- Stakeholder & Functional Requirements — new requirements added with unique IDs.
- Safety Considerations — new hazards, mitigations, and derived safety requirements.
- Subsystem Designs — mechanical, electrical, software, and control updates.
- Interfaces — any new mechanical, electrical, or communication interfaces.
- Design Rationale — justification for new design decisions.
- Verification & Validation Plan — new test cases and acceptance criteria.
- Compliance Matrix — updated mapping from requirements to tests.
- Configuration & Version Control — updated version entry documenting the change.

6.2 Updating Existing Documentation

When modifying an existing component, the Design Report **MUST** be updated in the following way:

- Requirements **MUST** be revised or added when functionality changes.
- Architecture and subsystem diagrams **MUST** be updated to reflect new behaviour.
- Test Cases and Acceptance Criteria **MUST** be updated to match new requirements.
- Compliance Matrix **MUST** be updated with new test results.
- Risks & Mitigations **MUST** be re-evaluated when design changes introduce new hazards.
- Version Control Table **MUST** include a new entry describing the change, date, and author.

The template explicitly supports this workflow through sections such as Verification & Validation Plan, Compliance Matrix, and Configuration & Version Control.

6.3 Versioning Rules

All documentation **MUST** follow a Git-style semantic versioning scheme:

- MAJOR - breaking changes to architecture, interfaces, or safety behaviour.
- MINOR - new features, new requirements, or new test cases.
- PATCH - small corrections, typo fixes, or clarifications.

Each version increment MUST be recorded in the Version Control table included in the template (e.g., “Use a Git-style versioning scheme (e.g., major.minor.patch)”).

6.4 Changelog Expectations

Every change MUST be recorded in the Design Report’s version table with:

- Version number
- Date
- Author(s)
- Summary of changes

Summaries MUST be concise but descriptive enough to understand what changed without reading the entire document. Examples include:

- “Updated electrical interface diagram”
- “Added safety requirement REQ-014”
- “Revised test cases for subsystem control logic”

This ensures traceability and supports audits, reviews, and integration.

6.5 Documentation Storage Requirements

All documentation MUST be stored in the google drive using the following structure:

- /Your-sub-section/Subcomponent/Design Report

Documents MUST NOT be stored locally or in personal drives.

The Design Report MUST always be accessible to the entire engineering team.

6.6 Documentation Quality Requirements

All documentation MUST meet the following quality standards:

- Written clearly and professionally.
- Free of ambiguous language.
- Uses consistent terminology across mechanical, electrical, software, and control sections.
- Includes diagrams where required by the template (e.g., System Overview, Architecture).
- Uses requirement IDs, test IDs, and risk IDs consistently.
- Fully completes all mandatory sections of the Design Report Template.

The template explicitly supports this by providing structured tables, section prompts, and examples (e.g., “List the functional expectations...”, “Define the measurable conditions...”).

6.7 Documentation as a Living Artefact

Documentation MUST be updated continuously throughout the design lifecycle.

A Design Report is NOT a final deliverable—it is a living document that evolves as:

- Requirements change
- Risks are discovered
- Interfaces evolve
- Tests are added
- Failures occur and are resolved

The Verification & Validation Plan, Compliance Matrix, and Version Control sections ensure this continuous update cycle is maintained.

7. Versioning & Release Standards

7.1 Semantic Versioning Rules

All software **MUST** use semantic versioning in the format:

MAJOR.MINOR.PATCH

MAJOR

Increment when:

- Interfaces change
- Behaviour changes in a way that breaks compatibility
- Architecture or communication protocols change
- Other components must be updated to remain compatible

MINOR

Increment when:

- New features are added
- New CAN objects, states, or behaviours are introduced
- Functionality expands without breaking existing behaviour

PATCH

Increment when:

- Bugs are fixed
- Minor improvements are made
- Documentation or comments are updated
- Internal refactors occur with no behavioural change

Hotfixes

- Hotfixes **MUST** increment the PATCH version.
- Hotfixes **MUST** be tagged immediately after merging.
- Hotfix branches **MUST** be merged back into both main and dev.

7.3 Branching Standards (General)

All repositories **MUST** follow a structured branching model:

- main — stable, production-ready code
- dev — integration branch for upcoming releases
- feature/ — short-lived branches for new work
- hotfix/ — urgent fixes branched from main

General rules:

- No direct commits to main or dev.
- All work **MUST** occur in feature or hotfix branches.
- All merges **MUST** undergo review.
- All merges **MUST** pass CI.
- Branch names **MUST** be descriptive and follow the naming conventions defined for the subsystem.

7.4 Node Software Release & Branching Standards

7.4.1 Branch Structure for Node Software

Main

- Contains the most up-to-date, stable, production-ready node software.
- Only Section Leads may approve merges into main.
- Every merge MUST correspond to a tagged release.
- No direct commits are allowed.

Dev

- All features MUST be merged into dev first.
- dev is the integration branch where node interactions are tested together.
- Section leads may cut you off from merging to dev if a new main release is upcoming.
- Section Leads coordinate integration timing to avoid conflicts between nodes.

feature branches

Feature branches MUST follow the naming convention:

NodeName/feature/short-description-label

Examples:

- motor_controller/feature/speed-control
- brakes/feature/brake-lockout

Rules:

- MUST branch from dev.
- MUST be focused on a single feature or requirement.
- MUST be merged into dev only after review and passing tests.
- MUST be deleted after merging.

hotfix branches

Hotfix branches MUST follow:

NodeName/hotfix/issue-description

Rules:

- MUST branch from main.
- MUST be reviewed by a Section Lead.
- MUST be merged into both main and dev.
- MUST be tagged immediately after merging.

7.4.2 Release Approval for Node Software

Node software releases MUST follow strict approval rules:

- Only Section Leads may approve merges into main.
- Section Leads MUST coordinate release timing across all nodes.
- Developers MUST notify a Section Lead before merging into dev.
- No release may proceed without:
 - Passing CI
 - Updated changelog
 - Updated version number
 - Updated Design Report version table
 - Updated test cases and compliance matrix (if behaviour changed)

7.4.3 Changelog Requirements for Node Software

Node software MUST maintain a high-quality changelog. You must follow the <https://keepachangelog.com/en/1.1.0/> Guidelines to keep this changelog.

7.4.4 Release Predictability Requirements

To maintain predictable releases across all nodes:

- No unplanned merges into dev or main.
- No feature may bypass dev.
- No release may occur without Section Lead coordination.
- All nodes MUST remain compatible with each other's CAN behaviour.
- All releases MUST be documented in the Design Report.

Standards and Guidelines Version Control

Any changes to the Standards / Guidelines Document MUST be documented here and the version should be reflected on the first page in its respective area. The versioning should follow the standard Git style Major.Minor.Patch framework.

Version	Update description	Date	Author/s
1.0.0	Basic Version of Standards	28/02/26	Chiara Gillam
1.0.1	Section 1 rehailed from feedback of software team	27/03/26	Chiara Gillam

Appendices (Templates, Examples, Checklists)